

Embedded Machine Learning Implementation Using eiQ Time Series on the i.MX RT1052 Kit with Sensor Data

1. Objective

This report aims to provide a technical and instructional guide for implementing embedded machine learning models using the **eiQ Time Series** tool developed by NXP. The practical application is carried out on the **i.MX RT1052** development kit, with input data collected from two sensor sources:

- **MPU6050 accelerometer** (external), configured for vibration analysis.
- **Onboard microphone**, configured for acoustic signal capture.

The focus is on creating and deploying classification or anomaly detection models, using real-world data processed and executed directly on the microcontroller.

2. Methodology

The complete development workflow for implementing embedded machine learning models using sensor data is described below:

2.1 Process Steps

- **Data Logging:**
The firmware running on the i.MX RT1052 kit is capable of capturing two types of signals:
 - **Accelerometer Data:** Acquired from the external **MPU6050** sensor ($\pm 4g$ range), sampled at **2 kHz**, and stored as **.csv** files containing sequential X, Y, Z values per line.
 - **Microphone Data:** Acquired using the onboard microphone, sampled at **16 kHz / 16-bit resolution**, and saved in both **.wav** and **.csv** formats for compatibility

with **eiQ Time Series**.

- **Data Preprocessing:**

Files are preprocessed using the **Data Operations** section within eiQ Time Series. Operations may include normalization, filtering, and sample alignment.

For accelerometer signals:

- According to the Nyquist Theorem, only frequency components **below 1000 Hz** should be analyzed (sampling rate = 2 kHz).
- The $\pm 4g$ range is suited for light to moderate vibrations. For high-impact detection, a wider range ($\pm 8g$ or $\pm 16g$) should be configured.

- For microphone signals:

- The signal is sampled at 16 kHz, allowing analysis of frequencies up to 8 kHz.
- These recordings are well-suited for audio classification, acoustic anomaly detection, or condition monitoring.

- Signal acquisition window size (e.g., 256 samples per line for accelerometer data) and duration (5 seconds) can be adjusted in firmware to better fit the target use case.

- **Dataset Creation:**

Two datasets should be prepared for each application:

- One representing normal signal behavior.
- One representing anomalous or multi-class behavior.

- **Training and Benchmarking:**

The **eiQ Time Series** platform automatically benchmarks multiple algorithms, evaluating performance based on accuracy, model size, and memory footprint to identify the best candidate for deployment.

- **Emulation:**

Trained models are validated using test data not included in the training set. This step helps assess generalization performance.

- **Deployment:**

The final model is exported as a precompiled library (**libtss.a**) and linked to the firmware application on the i.MX RT1052.

3. Configuration

1. Clone the project repository:
https://github.com/jvmreis/nxp_embedded_machine_learning
2. Open **MCUXpresso IDE**.
3. Navigate to **File > Import Project from File System**.
4. Select the cloned project directory (e.g.,
[Github\nxp_embedded_machine_learning](#)).
5. Configure the project to link the precompiled libtss.a library:
 - a. The **libtss.a** file generated during the **Deployment** phase in eiQ Time Series must be correctly linked into your firmware project. Follow the steps below:
 - b. Right-click on the project in the **Project Explorer**, then select **Properties**.
 - c. Navigate to:
C/C++ Build > Settings > MCU Linker > Libraries
 - d. In "**Libraries (-l)**", add: **tss**
 - e. In "**Library search path (-L)**", add:
`${workspace_loc}/${ProjName}/source/lib}`

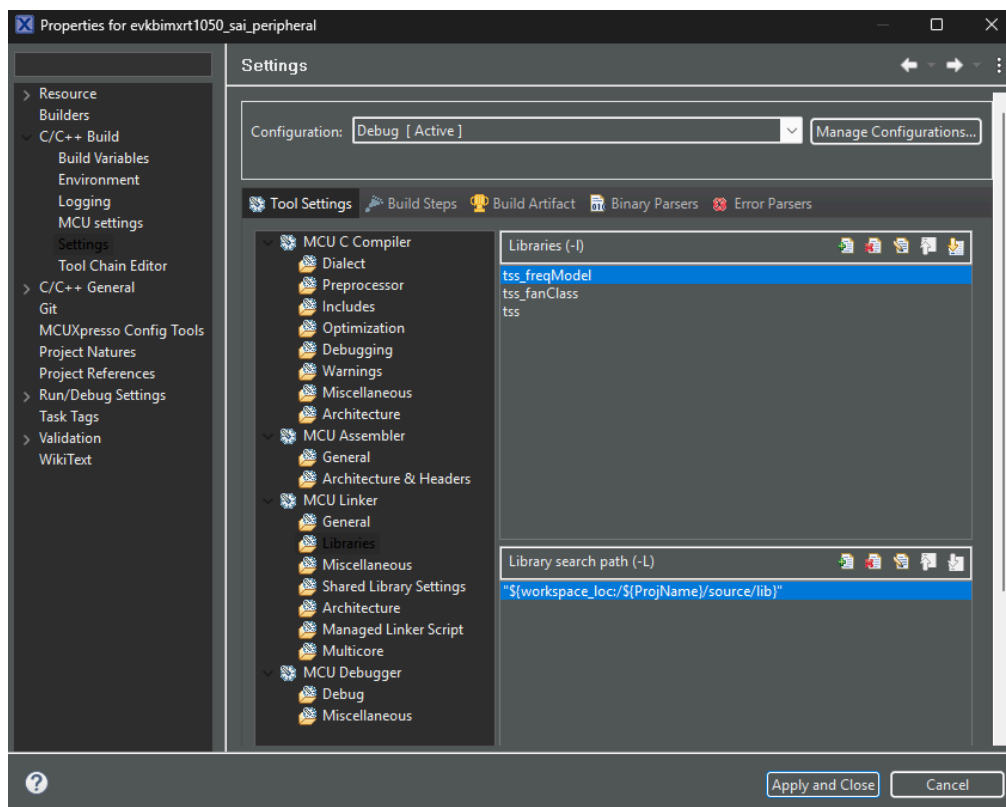


Figure 1 – Linker Configuration in MCUXpresso IDE for the **tss** Library

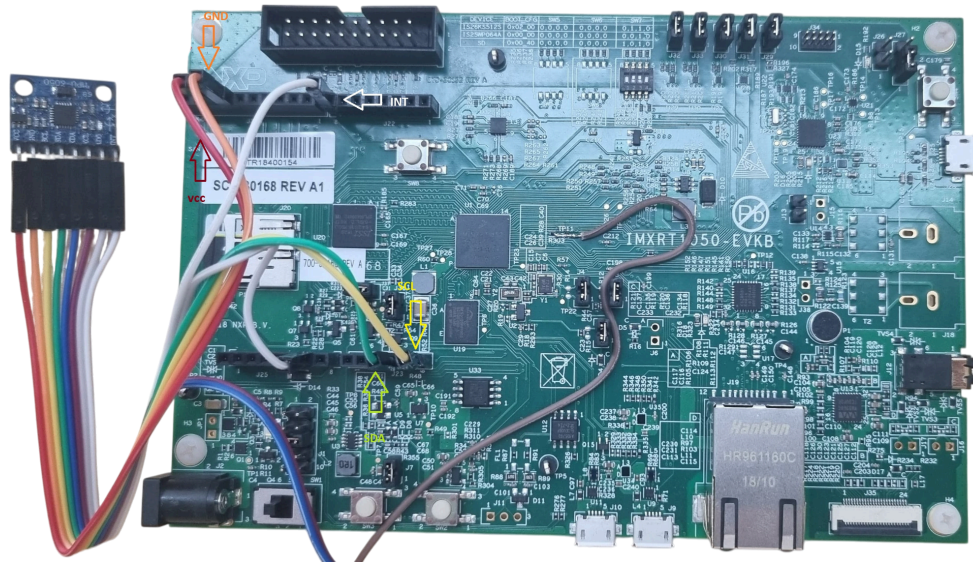
6. Click **Build** to compile the project.

3.1 Tools Used

- **Platform:** NXP i.MX RT1052
- **IDE:** MCUXpresso IDE
- **Serial Terminal:** Tera Term
- **AI Tool:** eiQ Time Series
- **Acceleration Sensor:** MPU6050
- **Storage Medium:** SD Card

This code is using an **external MPU6050** accelerometer, connect it to the development board using the following pin headers. If instead you prefer using the **on-board FXOS8700CQ accelerometer**, you will need to modify the firmware accordingly.

Signal	Pin Header	Pin Number
VCC	J24	Pin 8
GND	J24	Pin 7
INT	J22	Pin 8
SDA	J23	Pin 9
SCL	J23	Pin 8



4. Practical Experiment Guide

4.1 Data Logging

1. Connect to the i.MX RT1052 board using **Tera Term** at **115200 baud**.
2. Insert an SD card.
3. At startup, the terminal displays the following menu:

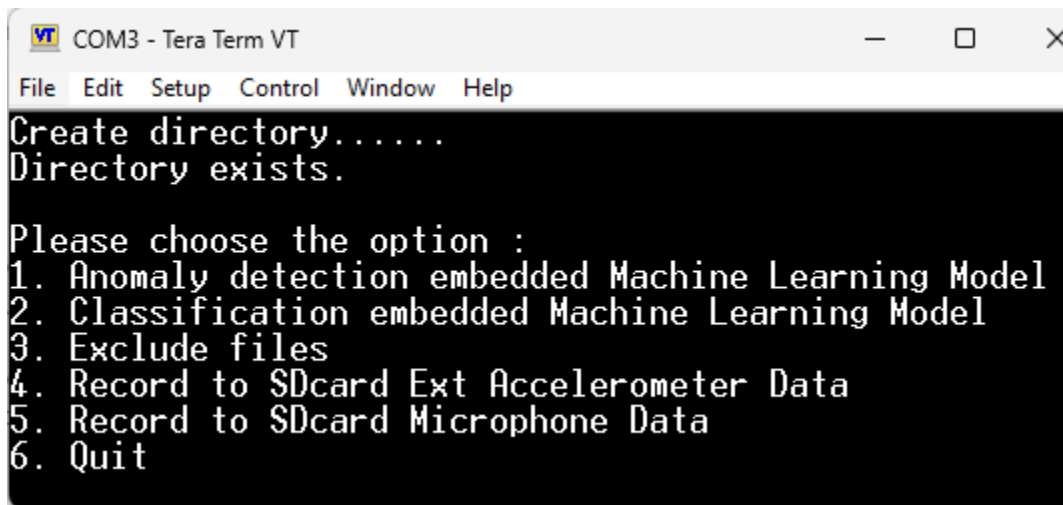


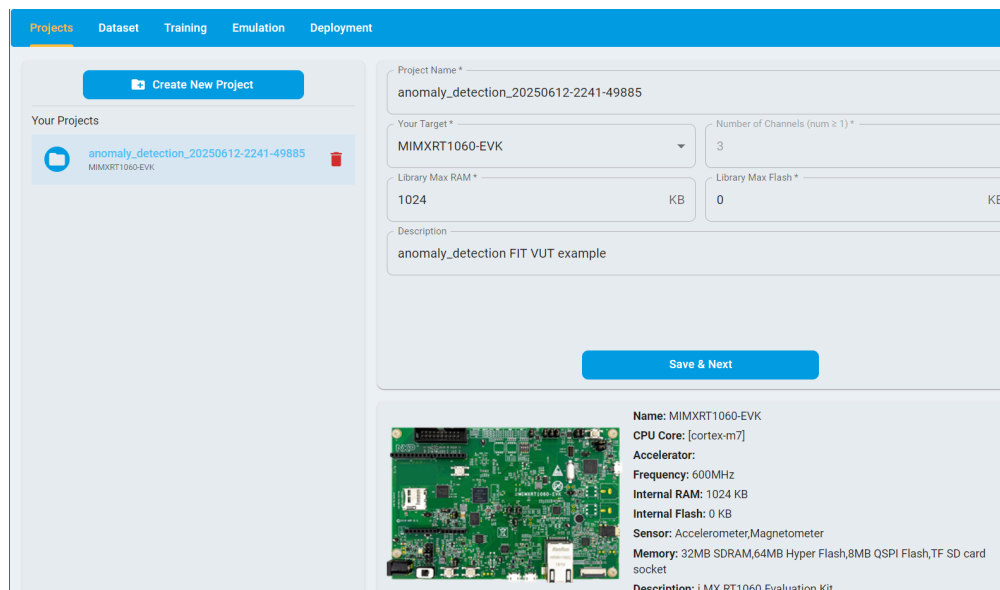
Figure 3 – Menu inference via serial terminal.

- **Option 4:** Captures 5 seconds of acceleration data from MPU6050. Stored as **.csv** (e.g., **ACCELE_1.csv**).
- **Option 5:** Captures 5 seconds of audio data from the onboard microphone. Stored as both **.wav** and **.csv**.

Use both options to collect signals for each class (normal, anomalous).

4.2 Model Training in eiQ Time Series

- Open **eiQ Time Series**.
- Choose the desired model type: **Classification** or **Anomaly Detection**.
- Create a new project with the following settings:
 - Platform:** i.MX RT1050
 - Number of Channels:** 3 (X, Y, Z)
 - RAM/Flash:** Default (or customized, if required)



The screenshot displays the 'Create New Project' interface in the eiQ software. The 'Project Name' is 'anomaly_detection_20250612-2241-49885'. The 'Your Target' is set to 'MIMXRT1060-EVK'. The 'Number of Channels (num ≥ 1)' is set to '3'. The 'Library Max RAM' is '1024 KB' and the 'Library Max Flash' is '0 KB'. The 'Description' is 'anomaly_detection FIT VUT example'. A 'Save & Next' button is visible. Below the settings, there is a hardware specification section for the MIMXRT1060-EVK, including CPU Core (cortex-m7), Accelerator, Frequency (600MHz), Internal RAM (1024 KB), Internal Flash (0 KB), Sensor (Accelerometer, Magnetometer), Memory (32MB SDRAM, 64MB Hyper Flash, 8MB QSPI Flash, TF SD card socket), and Description (i.MX RT1060 Evaluation Kit).

Figure 4 – *eiQ project creation screen with model type and hardware settings.*

Upload the corresponding .csv files with the collected signals.

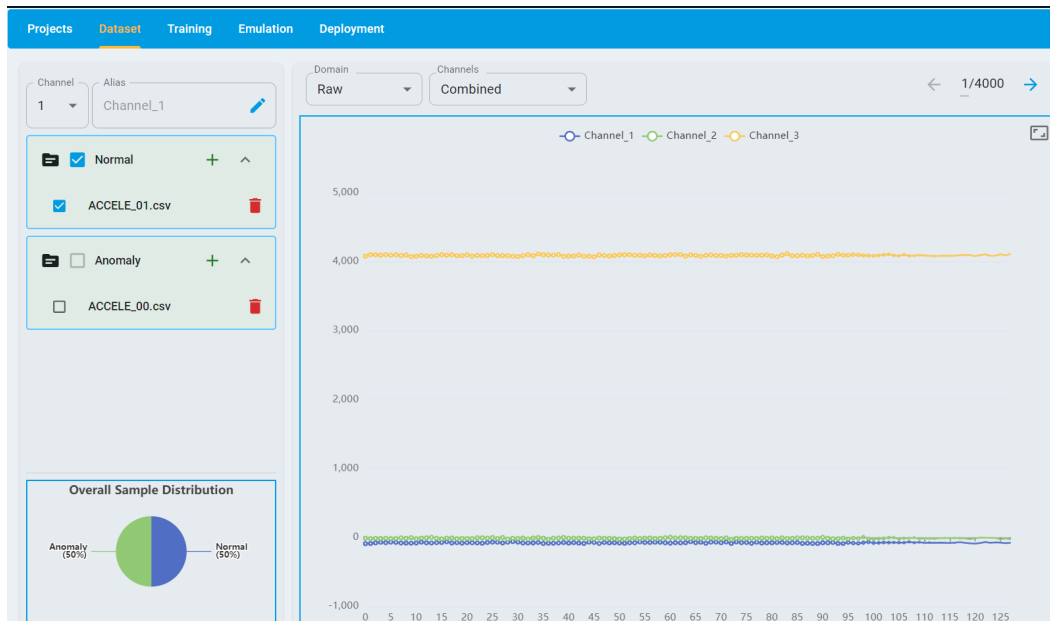


Figure 5 – eiQ project Dataset creation screen with data upload

4.3 Training and Benchmarking

1. Navigate to the **Training** tab.
2. Click **Start New Training**.
3. Wait for the automatic benchmarking process, where different algorithms are tested. The tool will recommend the best-performing model based on accuracy and footprint.

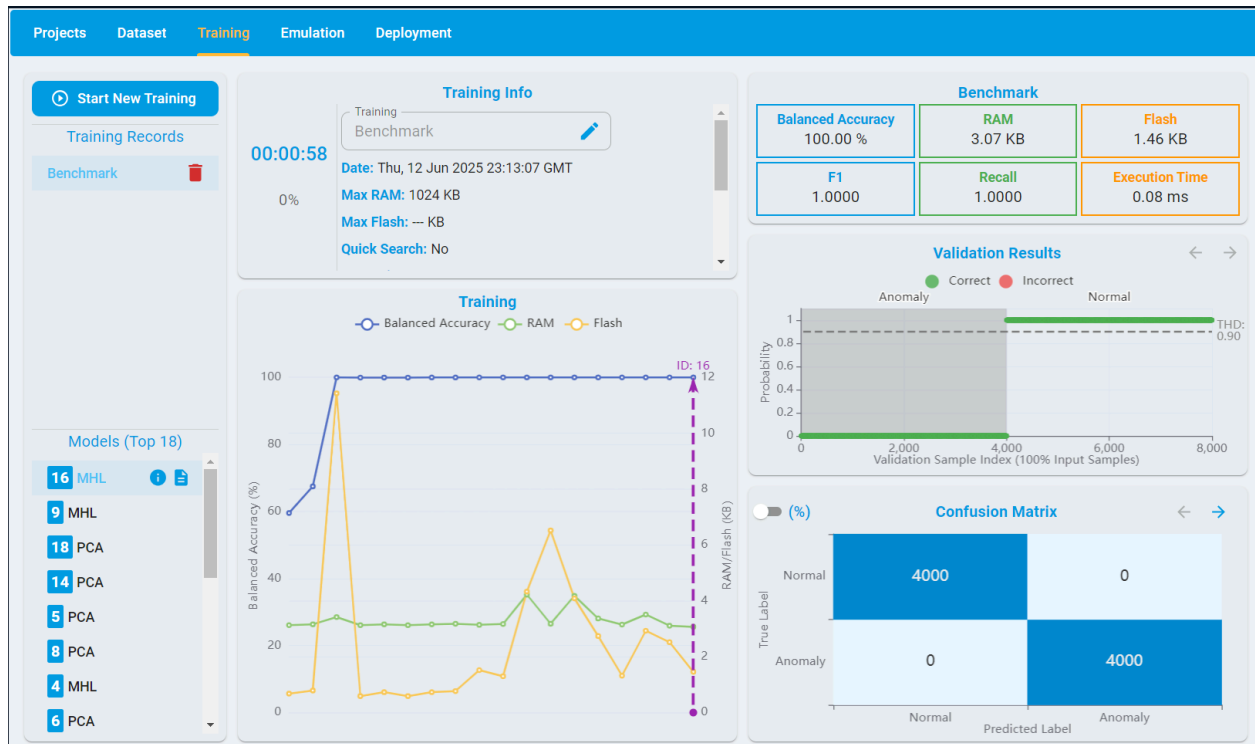


Figure 6 – Automatic benchmarking output and model selection result.

4.4 Emulation

1. Select the trained model.
2. Go to the **Emulation** tab.
3. Upload test signals not used during training.
4. Analyze the model's output based on similarity with known patterns.

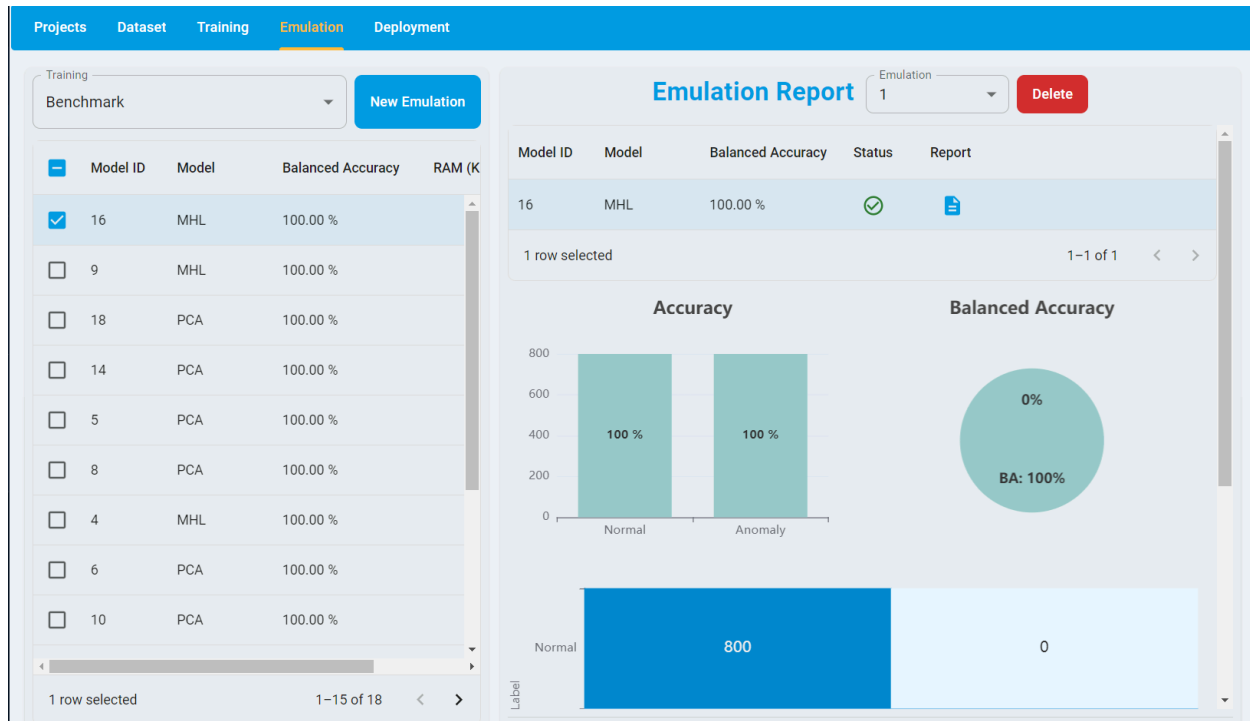


Figure 7 – Emulation results showing signal similarity and inference response.

4.5 Deployment to the Microcontroller

Once the training process is complete in **eiQ Time Series**, the selected model can be exported by clicking **Generate** in the **Deployment** tab. This action generates a precompiled static library (**libtss.a**) that must be integrated into the firmware running on the i.MX RT1052 development board.

4.5.1 Steps to Deploy and Run the Model:

1. Extract the contents of the exported **.zip** package and place the **libtss.a** file into the project directory at:
`C:\Github\nxp_embedded_machine_learning\source\lib`
2. In **MCUXpresso IDE**, open the project settings:
 - Right-click the project → **Properties**
 - Go to: **C/C++ Build > Settings > MCU Linker > Libraries**

- Add `tss` in **Libraries (-I)**
 - Add `${workspace_loc}/${ProjName}/source/lib` in **Library search path (-L)**
3. Rebuild the project and flash the updated firmware to the i.MX RT1052 using the debugger or USB flasher.
 4. Open **Tera Term** (or another serial terminal) and connect to the board at **115200 baud**. The following menu will appear as the figure 3.
 - **Option 1:** Starts real-time inference using the **anomaly detection model**.
 - **Option 2:** Starts real-time inference using the **classification model**.

4.5.2 Selecting the Sensor Input (Accelerometer or Microphone):

The input source for the machine learning model can be configured in the file `machine_learning.c`.

Inside this file, two core functions are used to collect input data:

- `acc_sample_data(float *data_buffer)` — captures one window of acceleration data (X, Y, Z axes).
- `mic_sample_data(float *data_buffer)` — captures one window of raw audio data (512 samples at 16 kHz, 16-bit).

To change the input source used for inference, simply modify the function call inside either:

- `ml_anomaly_detection()` (uses accelerometer by default)
- `ml_classification()` (uses microphone by default)

For example, to test classification based on accelerometer instead of microphone, you could replace:

```
mic_sample_data(mic_data_input);

status = tss_cls_predict_freqModel(mic_data_input, probabilities,
&class_index);
```

```
acc_sample_data(acc_data_input);

status = tss_cls_predict_freqModel(acc_data_input, probabilities,
&class_index);
```

This allows developers to flexibly switch between sensor modalities depending on the application—whether it's vibration analysis or sound-based detection.

4.5.3 Default configuration

By default, the embedded firmware is pre-configured to demonstrate two machine learning use cases:

- **Anomaly Detection Model (Option 1 in UART menu):**
Uses the **external accelerometer (MPU6050)** to distinguish between two states:
 - **Idle sensor** (stationary condition)
 - **Shaken sensor** (induced vibration or movement)
- **Classification Model (Option 2 and 3 in UART menu):**

Use option 2 to run the **onboard microphone** to classify **sine wave sounds** at different frequencies. The model outputs probabilities.

Use option 3 to run the **external accelerometer** to classify **Fan** at different states (on fan, off fan, friction and clogged). The model outputs probabilities.

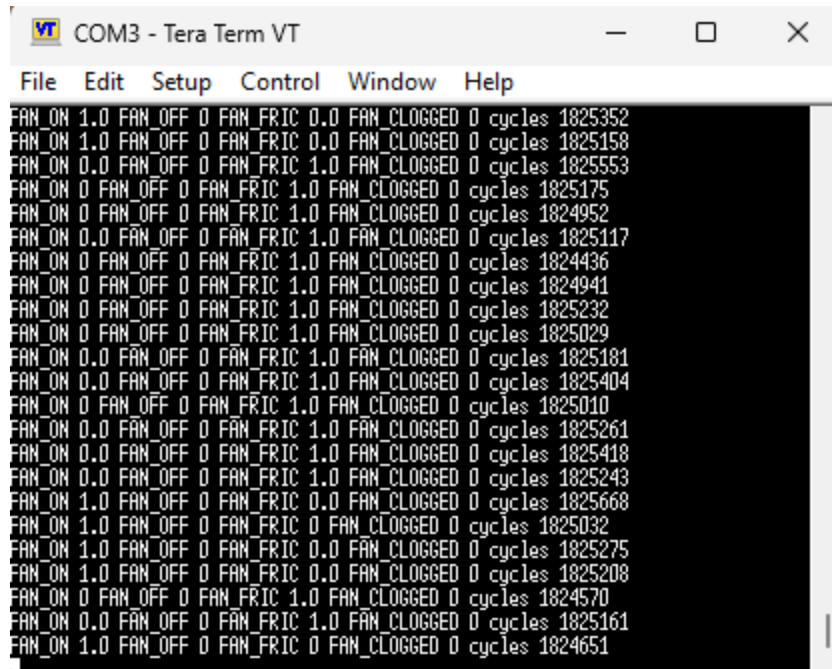
These configurations are intended for demonstration and testing purposes and can be customized in the `machine_learning.c` file to suit other sensor data or classification targets.

COM3 - Tera Term VT				
File	Edit	Setup	Control	Window Help
anomaly detection	ion	1.000000	cycles	3911
anomaly detection	ion	1.000000	cycles	3918
anomaly detection	ion	1.000000	cycles	3913
anomaly detection	ion	1.000000	cycles	3913
anomaly detection	ion	1.000000	cycles	3912
anomaly detection	ion	1.000000	cycles	3919
anomaly detection	ion	1.000000	cycles	3913
anomaly detection	ion	1.000000	cycles	3914
anomaly detection	ion	1.000000	cycles	3912
anomaly detection	ion	1.000000	cycles	3913
anomaly detection	ion	1.000000	cycles	3915
anomaly detection	ion	1.000000	cycles	3916
anomaly detection	ion	1.000000	cycles	3914
anomaly detection	ion	1.000000	cycles	3909
anomaly detection	ion	1.000000	cycles	3916
anomaly detection	ion	1.000000	cycles	3914

Figure 8 – Embedded anomaly detection model with accelerometer signal running

COM3 - Tera Term VT									
File	Edit	Setup	Control	Window	Help				
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 194528
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 194935
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195091
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195452
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 195931
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195524
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 194910
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 194854
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195122
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195467
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195019
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 194568
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 195353
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 195732
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195187
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 194846
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 194724
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 195034
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195323
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 195174
4khz	0	2khz	0	1780hz	1.0	860hz	0	244hz	0 cycles 195314
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 195363
4khz	0	2khz	0	1780hz	0.8	860hz	0	244hz	0.2 cycles 195389
4khz									

Figure 9 – Embedded Classification frequency model with microphone signal running



```
VT COM3 - Tera Term VT
File Edit Setup Control Window Help
FAN_ON 1.0 FAN_OFF 0 FAN_FRIC 0.0 FAN_CLOGGED 0 cycles 1825352
FAN_ON 1.0 FAN_OFF 0 FAN_FRIC 0.0 FAN_CLOGGED 0 cycles 1825158
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825553
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825175
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1824952
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825117
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1824436
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1824941
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825232
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825029
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825181
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825404
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825010
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825261
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825418
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825243
FAN_ON 1.0 FAN_OFF 0 FAN_FRIC 0.0 FAN_CLOGGED 0 cycles 1825668
FAN_ON 1.0 FAN_OFF 0 FAN_FRIC 0 FAN_CLOGGED 0 cycles 1825032
FAN_ON 1.0 FAN_OFF 0 FAN_FRIC 0.0 FAN_CLOGGED 0 cycles 1825275
FAN_ON 1.0 FAN_OFF 0 FAN_FRIC 0.0 FAN_CLOGGED 0 cycles 1825208
FAN_ON 0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1824570
FAN_ON 0.0 FAN_OFF 0 FAN_FRIC 1.0 FAN_CLOGGED 0 cycles 1825161
FAN_ON 1.0 FAN_OFF 0 FAN_FRIC 0 FAN_CLOGGED 0 cycles 1824651
```

Figure 10 – Embedded Classification Fan state model running

7. Reference

NXP RM – i.MX RT1050 Processor Reference Manual, section 51.5.1.1 (Restart Mode)

<https://www.nxp.com/webapp/Download?colCode=IMXRT1050RM>

Github Repository

https://github.com/jymreis/nxp_embedded_machine_learning